

PARADIGMA ORIENTADO A OBJETOS

UNIDAD N° I

Objetos en el Mundo



SEMANA 2

Consideraciones previas

El contenido que se expone a continuación está ligado a los siguientes objetivos:

- Distingue los diversos niveles de acceso aplicables a clases, métodos y atributos de la clase.
- Distingue los diversos tipos de datos aplicables a métodos y atributos dentro de una clase.
- Distingue las diferencias entre los términos clases, objetos e instancia dentro de la programación orientada a objetos.
- Distingue las propiedades o atributos de una clase, incluyendo sus métodos acceso y mutación según requerimientos.
- Distingue el método constructor como el método principal de la clase y otros tipos de métodos.
- Aplica los nuevos conceptos aprendidos de la orientación a objetos para la construcción de aplicación.

Sobre las fuentes utilizadas en el material

El presente Material de Estudio constituye un ejercicio de recopilación de distintas fuentes, cuyas referencias bibliográficas estarán debidamente señaladas al final del documento. Este material, en ningún caso pretende asumir como propia la autoría de las ideas planteadas. La información que se incorpora tiene como única finalidad el apoyo para el desarrollo de los contenidos de la unidad correspondiente, respetando los derechos de autor ligados a las ideas e información seleccionada para los fines específicos de cada asignatura.

Introducción

Esta es la segunda semana del curso, y en esta ocasión se comenzarán a ver más en detalle los conceptos asociados a la programación a objetos. Se abordará el concepto de modificador de acceso, los diferentes tipos de datos aplicables a atributos, métodos y clases, y el concepto de accesor y mutador. Asimismo, se conocerá y analizará el concepto de constructor, y su relevancia en la programación orientada a objetos.

Otros conceptos que serán abordados en esta semana son los parámetros que pueden recibir los métodos de una clase, y los valores de retorno que éstos pueden devolver.

Se hace hincapié en que, a partir de esta semana, los ejercicios que se realizarán tendrán una mayor cantidad de código. Esto significa que, si bien se seguirán utilizando las opciones que entrega Greenfoot, se combinarán con más código Java nativo.

Ideas fuerza

1. **Modificador de acceso:** son palabras clave que se usan para especificar la accesibilidad declarada de un miembro o un tipo. Son aplicables a clases, métodos y atributos.
2. **Tipo de dato:** es una clasificación que se puede otorgar a un elemento, y determina los valores posibles que podrá tener y, por consiguiente, el espacio reservado para almacenar datos.
3. **Accesor:** llamados también “getters”, son métodos que permiten obtener el valor de un atributo de un objeto. No reciben parámetros, y retornan un tipo de dato similar al atributo al que hacen referencia.
4. **Mutador:** llamados también “setters”, son métodos que permiten modificar el valor de un atributo de un objeto. No retornan valor alguno, y reciben como parámetro y valor con un tipo de dato igual al atributo al que hacen referencia.
5. **Constructor:** son métodos que permiten inicializar los atributos de una clase al momento de crear una nueva instancia. Pueden o no recibir parámetros, y una clase puede tener más de un método constructor, siempre y cuando no reciban la misma cantidad de atributos con los mismos tipos de datos.
6. **Retorno:** se conoce como valor de retorno a la variable que se devuelve al momento de concluir la ejecución de un método. No es obligatorio que un método tenga un retorno, y el tipo de dato que se retorna se establece junto al nombre del método.

Índice

Contenido

<i>1.- Modificadores de acceso.....</i>	<i>6</i>
1.1.- public	6
1.2.- private.....	6
1.3.- protected.....	6
<i>2.- Tipos de datos.....</i>	<i>7</i>
2.1.- String.....	8
2.2.- int	10
2.3.- boolean	10
2.4.- Object	10
2.5.- char	11
2.6.- Date	11
2.7.- Otros tipos.....	12
<i>3.- Clases, objetos e instancias.....</i>	<i>12</i>
<i>4.- Propiedades de las clases</i>	<i>13</i>
4.1.- Getter.....	13
4.2.- Setter	14
<i>5.- Métodos</i>	<i>15</i>
5.1.- Constructores	16
5.2.- Método con o sin parámetros	16
5.3.- Métodos nativos	17
5.4.- Directiva return	17

Desarrollo

1.- Modificadores de acceso

Los modificadores de acceso o visibilidad se refieren al nivel de accesibilidad de los atributos y métodos [1]. Los tipos que existen son: public, private y protected.

1.1.- public

Los métodos de tipo “public” se pueden usar en cualquier otra clase y, por lo tanto, son los que definen la funcionalidad de la clase [2]. En el caso de los atributos, al ser de este tipo podría ser accedido al instanciar un objeto, pero esto es un comportamiento que no debería darse.

1.2.- private

Los métodos private son aquellos que sólo se pueden usar en el código que se escriba dentro de la propia clase en la que se declara. Son métodos auxiliares que sirven únicamente como apoyo al desarrollo de la propia clase.

A nivel de atributos, uno de tipo private solo será accesible dentro de la clase. Por lo general, todos los atributos deberían ser de tipo private, ya que las características de un objeto deberían ser solo manipulables internamente, ya sea accediendo a sus valores o bien modificándolos.

Las clases del API de Java también contienen métodos privados, pero no se muestran en la documentación. La API de Java, en tanto, es la interfaz de programación de aplicaciones (de ahí la sigla) que fue creada por los desarrollos del lenguaje; dicho de otro modo, es el conjunto de clases y métodos que facilitan la labor del desarrollador.

1.3.- protected

El nivel de acceso `protected` para los efectos del aprendizaje de Java como lo estamos analizando actualmente es similar al nivel público. La diferencia entre `protected` y `public` se establece a nivel de paquete.

Un paquete es un conjunto de elementos dentro de un proyecto Java. En lo concreto se traducen en carpetas o directorios. Sin embargo, en el contexto de Greenfoot esto no se aplica.

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

Todo sería más fácil si se declaran todos los métodos y atributos como “public”. Sin embargo ¿por qué no sería recomendable proceder de esta manera?



2.- Tipos de datos

Todo programa requiere la creación de variables, las que permitirán el almacenamiento de información temporal en memoria dinámica. Cada una de las variables creadas, deben tener características definidas por tipos de datos para poder almacenar información.

Atendiendo al tipo de información que representan, los tipos de datos y las variables de ellos definen se pueden clasificar en:

- **Tipos elementales:** se denominan también básicos o primitivos, y son los tipos de variables que no se establecen o construyen a partir de otros, y su representación y operaciones vienen dadas por el propio lenguaje de programación.
- **Tipos referencia:** llamados también complejos o estructurados; son aquellos tipos de datos que se construyen por agregación de datos que pueden ser del mismo o de distinto tipo; pueden venir predefinidos en el lenguaje, normalmente en forma de librerías, o pueden ser definidos por el programador.

En las siguientes tablas se detallan los tipos de datos elementales y de referencia que establece el lenguaje de programación Java.

Tipos de datos	Clasificación	Identificador de tipo en Java
Enteros	Elemental	byte, short, int, long
Reales	Elemental	float, double
Caracteres	Elemental	char
Lógicos	Elemental	boolean
Clases estándar	Referencia	String, System, Scanner, Integer, ...
Clases de usuario	Referencia	Auto, Edificio

2.1.- String

Los "String" son objetos de java con una sintaxis especialmente cómoda para representar cadenas de caracteres. Los caracteres se codifican usando Unicode (Unicode es un sistema de codificación de caracteres utilizado por los equipos informáticos para el almacenamiento y el intercambio de datos en formato de texto). Java permite escribir directamente cadenas entre comillas.

```
String autor = "Miguel de Cervantes";  
String colega = "Doménikos Theotokópoulos, "El Greco";
```

Para incluir el carácter comillas dobles en una cadena, escriba "". Una operación interesante entre Strings es la concatenación:

```
String nombre = "Miguel";  
String apellido = "Muñoz";
```

```
String colega = nombre + " " + apellido;
```

La clase String define muchos métodos interesantes para hacer programas:

Método	Explicación
int length()	Devuelve el número de caracteres de una cadena
boolean equals(String b)	Determina si la cadena contiene los mismos caracteres que la cadena "b".
boolean equalsIgnoreCase(String b)	Determina si la cadena contiene los mismos caracteres que la cadena "b", independientemente de que estén en mayúsculas o minúsculas.
int compareTo(String b)	Compara contra la cadena "b", devolviendo: · un valor negativo si la cadena es anterior a "b"; · cero (0) si la cadena es igual a "b"; · un valor positivo si la cadena es posterior a "b". En las comparaciones se usa el código Unicode, lo que no siempre responde al orden lexicográfico (de diccionario).
String trim()	Crea un nuevo objeto eliminado el espacio en blanco que pudiera haber al principio o al final.
char charAt(int posicion)	Extrae un carácter en la posición indicada. Es importante recordar que la primera posición es la 0, mientras que la última es n-1, donde n es la cantidad de caracteres del String.
String substring(int a, int z)	Extrae la subcadena entre las posiciones a y z
String substring(int desde)	Extrae la subcadena desde la posición indicada
int indexOf(char carácter) int indexOf(String cadena)	Indica en qué posición se encuentra el carácter (o cadena) indicado por primera vez, buscando desde el principio.
int lastIndexOf(char carácter) int lastIndexOf(String cadena)	Indica en qué posición se encuentra el carácter (o cadena) indicado por primera vez, buscando desde el final
boolean startsWith(String prefijo)	Retorna verdadero ("true") si la cadena comienza con el prefijo indicado, y falso en caso contrario.
boolean endsWith(String sufijo)	Retorna verdadero ("true") si la cadena termina con el sufijo indicado, y falso en caso contrario.

2.2.- int

El tipo de dato int permite almacenar números enteros, esto implica números positivos, negativos y el cero. Tiene un valor máximo y mínimo para definir valores; por lo mismo, si se desea definir variables que no llegarán a valores tan altos o pequeños, se puede usar otros tipos de datos similares. En la tabla siguiente se muestran los tipos de datos numéricos junto con su tamaño en bits y valores límite.

Nombre	Tamaño	Rango
byte	8 bits	[-128, 127]
short	16 bits	[-32768, 32767]
int	32 bits	[-2147483648, 2147483647]
long	64 bits	[-9223372036854775808, 9223372036854775807]

Otro aspecto importante a destacar, es que en las variables numéricas, independiente de su tipo, se puede aplicar las operaciones aritméticas comunes. En la tabla siguiente se muestra las operaciones en Java y el símbolo que corresponde.

Operador	Descripción
+	Suma o signo
-	Resta o signo
*	Multipliación
/	División
%	Módulo o resto de la división entera

2.3.- boolean

El lenguaje Java implementa un álgebra booleana bivaluada (con dos valores de verdad) mediante el tipo de datos boolean, Los dos únicos valores de verdad de este tipo se representan con las constantes “true” para el valor verdadero y “false” para el valor falso.

2.4.- Object

La Clase Object [3] es una clase especial definida en Java. Todas las demás clases son subclases de Object. Esto significa que una variable de referencia del tipo Object puede referirse a un objeto de cualquier otra clase.

2.5.- char

El tipo carácter se utiliza para representar letras latinas minúsculas y mayúsculas, números, signos de puntuación, caracteres de control y caracteres de distintos alfabetos como el griego, cirílico, hebreo, mandarín, árabe, etc.

Los literales de tipo carácter se escriben entre comillas simples, por ejemplo 'a', 'Z', '8', etc. Se representan internamente como valores enteros positivos, pero sin la representación en complemento a dos, ya que no se requieren valores negativos.

La asociación de cada carácter a un código numérico determinado se hace siguiendo algún estándar de codificación de caracteres que, en el caso del Java, es el denominado Unicode, en concreto UTF16; con esta codificación cada carácter ocupa habitualmente dos bytes (16 bits).

2.6.- Date

La clase Date representa un instante de tiempo específico con una precisión en milisegundos. Adicionalmente, la clase Date permite el uso del formato "Universal Coordinated Time", conocido por sus siglas UTC. Por otro lado, muchos computadores están definidos en términos de "Greenwich Mean Time", en sus siglas GMT, que es equivalente a Universal Time, UT. GMT es el nombre estándar y UT es el nombre científico del estándar. La diferencia entre UT y UTC es que UTC está basado en un reloj atómico y UT está basado en un reloj astronómico.

Las fechas en Java comienzan en el valor "standar based time" llamado "epoch" que hace referencia al 1 de enero de 1970, 0 horas 0 minutos 0 segundos GMT.

La clase Date posee métodos que permiten la manipulación de fechas. La clase Date pertenece al paquete java.util del API de Java.

2.7.- Otros tipos

Además de los tipos de datos que se han analizado, existe otro tipo importante de revisar. Estos son los números reales o decimales. Java dispone de los tipos “float” y “double” para trabajar con los números reales. La representación interna es la de punto flotante, diferente a la utilizada para los enteros, El conjunto de números reales es infinito no numerable y es imposible codificar todos los valores de un intervalo, teniendo que asumir una cierta imprecisión y trabajar con un número finito de valores.

Los números reales se caracterizan por dos magnitudes: la precisión y el intervalo de representación. La precisión es el número de dígitos significativos con los que se puede representar un número y el intervalo es la diferencia entre el mayor y el menor número que se pueden representar, La precisión de un número real depende del número de bits de su mantisa, mientras que el intervalo depende del número de bits de su exponente.

Así, el tipo “double” logra el doble de precisión que el “float”, entendiendo como precisión la cantidad de decimales. En estos tipos, lo más importante no es lo grande o pequeño que es el número para representar, sino su precisión.

En la siguiente tabla se analizan ambos tipos, su tamaño, rango y su precisión.

Nombre	Tamaño	Rango	Precisión
float	32 bits	[1.4E-45, 3.4028235E38]	7 decimales
double	64 bits	[4.9E-324, 1.7976931348623157E308]	15 decimales

3.- Clases, objetos e instancias

La definición de una clase Java se escribe en un archivo cuya extensión es .java. Su estructura general es la que se muestra a continuación:

```
[modificadores] class NombreDeLaClase [extends OtraClase] {  
    [[modificadores] tipo nomVar1;  
    [modificadores] tipo nomVar2;  
    ...  
    [modificadores] tipo nomVarN;]
```

```
[[modificadores] tipo nomMetodo1 ([listaParams]) { cuerpo }  
[modificadores] tipo nomMetodo2 ([listaParams]) { cuerpo }  
...  
[modificadores] tipo nomMetodoM ([listaParams]) { cuerpo }]
```

Este esquema donde los ítems que aparecen entre corchetes tienen un carácter opcional, permite observar claramente que la definición de una clase consta de dos partes bien diferenciadas: en la primera, se describen en Java (o declaran) los denominados atributos de la clase (nomVar1, ... , nomVarN), este es, los elementos distintivos de los que está compuesto (o que tiene) cualquier objeto de la clase; en la segunda se describen en Java (o declaran) los denominados métodos de la clase (nomMetodo1, ... , nomMetodoM), es decir, las operaciones que se pueden aplicar a cualquier objeto de la clase. Nótese que es posible que una clase puede no tener atributos y no tener métodos.

Cada nuevo elemento que se genere a partir de una clase es un objeto. Todos los objetos de la misma clase tendrán los mismos atributos y métodos, y todos tendrán la visibilidad que se haya definido en la clase. A un objeto ya inicializado no se puede agregar nuevos métodos o atributos, ya que rompería con la estructura original de la clase. Sin embargo, un objeto a partir del momento que es creado tiene vida propia, se puede modificar y acceder a sus atributos, e invocar al mismo tiempo a los métodos que posee.

Cuando se habla de instancia, en el contexto de la programación orientada a objetos, es un sinónimo de objeto.

4.- Propiedades de las clases

Tal como se mencionó antes, toda clase tiene un nombre, un alcance o ámbito, atributos y métodos. Ya se ha hablado anteriormente de los diferentes tipos de atributos que una clase puede tener y la estructura que toda clase debería poseer.

En esta sección se hablará respecto de métodos especiales de clases, que permiten obtener sus valores y modificar los datos ya existentes. A estos métodos se les denomina “getters” (accesores) y mutadores (setters).

4.1.- Getter

Un “getter” o accesor es un método que permite obtener el valor de un atributo. En muchas clases los atributos se declaran como privados, a fin de no poder acceder directamente a sus valores. El problema es que, si los atributos son privados ¿cómo podré conocer su valor actual?

Por convención, toda clase debe tener un método getter por cada atributo de la tabla. Cada getter debe retornar un valor del mismo tipo de dato del atributo al cual se asocia, y no necesitan parámetros de entrada para realizar sus acciones.

En cuanto a su alcance, deben ser métodos tipo “public”. Además, si los atributos de la clase no son privados, su inclusión se podría considerar opcional.

4.2.- Setter

Un “setter” o modificador o mutador es un método que permite cambiar el valor de un atributo en un objeto. Al igual que con los getters, si los atributos de la clase son privados, su incorporación es fundamental ya que, de otro modo, no se podría modificar valores en un objeto.

Por convención todo atributo debería tener un setter, el cual debe retornar un valor vacío o “void”, y recibir como parámetro una variable del mismo tipo que el atributo al que hace referencia el método.

Si los atributos de una clase son públicos, el incluir los “setters” podría considerarse una opción.

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

¿Es posible que una clase tenga un atributo con más de un getter o un setter?

Si los atributos de una clase son privados, y la clase no tiene setters, ¿cómo se puede asignar (inicialmente) valores a los objetos a partir de ella?



5.- Métodos

Los métodos de una clase Java definen las operaciones que se pueden aplicar a cualquiera de sus objetos. Se declaran, habitualmente, como public y, obligatoriamente, su definición consta de los siguientes ítems:

- A. **Cabecera o perfil:** en ella se detalla el nombre del método, el tipo del resultado que devuelve (return) y la lista de parámetros que pudiera requerir llevar a cabo la operación que representa. Es importante señalar ahora que un método Java puede no devolver resultado alguno, pues su labor pudiera ser exclusivamente la de modificar el estado del objeto sobre el que se aplica. Si un método no debe retornar un valor como resultado, el tipo de dato que devuelve será void.
- B. **Cuerpo:** en esta sección se detalla el código a ejecutar para obtener el resultado del método, Pueden formar parte de este código cualquier

instrucción incluida en el repertorio del lenguaje: asignación, composición, condicional, de repetición (bucle) y cualquier combinación de las anteriores; todos estos temas serán visto más adelante en este curso y en posteriores ocasiones. Es importante recordar también que, a menos que el tipo de retorno de un método sea void, la instrucción return debe aparecer obligatoriamente en el cuerpo de un método, pues su efecto es devolver su resultado.

A continuación, se indicarán los distintos tipos de métodos que se puede crear dentro de una clase Java.

5.1.- Constructores

Son los métodos que se usan para crear un objeto. Se distinguen del resto de los métodos de una clase por dos motivos:

- en su cabecera no existe o no se especifica el tipo de resultado
- su nombre debe ser igual que el de la clase

En el cuerpo del constructor deben existir las instrucciones de inicialización de los atributos de la clase, resultando más que recomendable que no exista ninguna más.

Otro aspecto importante es que un constructor puede recibir o no parámetros; si no recibe parámetros, los atributos se llenan con valores por defecto según su tipo. Además, una clase puede tener más de un constructor, siempre y cuando no existan dos constructores que reciban los mismos parámetros.

5.2.- Método con o sin parámetros

Tal como se mencionó antes, un método puede o no recibir parámetros. Los parámetros pueden ser de cualquier tipo, ya sean atributos primitivos o estructurados.

En cuanto a la cantidad de parámetros que un método puede recibir, no existe un límite establecido. Sin embargo, no es recomendable declarar parámetros en un método de manera indiscriminada, ya que su llamado sería complejo.

5.3.- Métodos nativos

Un método nativo [4] es un método Java (ya sea un método de instancia o un método de clase) cuya implementación está escrita en otro lenguaje de programación como C.

Los métodos nativos solo se deben usar en los casos en que Java puro no pueda responder a sus necesidades de programación.

Se debe limitar el uso de métodos nativos y emplearlos solo en las circunstancias siguientes:

- Para acceder a funciones del sistema que no están disponibles por medio de Java puro.
- Para implementar métodos sensibles al rendimiento que pueden beneficiarse en gran medida de una implementación nativa.
- Para intercambiar información con las interfaces de programación de aplicaciones (API) existentes que permitan a Java llamar a otras API.

5.4.- Directiva return

El tipo de retorno puede ser tanto un tipo primitivo como una referencia a objeto. Por ejemplo, el método `indexOf(char)` de `String` retorna un valor de tipo `int`, mientras que el método `valueOf(double)` de la misma clase retorna un objeto `String`. Cabe insistir en que la palabra reservada `void` indica que el método no devuelve ningún valor, como es el caso del método `gc()` de la clase `System`.

Existe una instrucción especial que se usa en el cuerpo de los métodos no `void` para darle valor a cada llamada, y cuya sintaxis es:

```
return expresion;
```

En el caso anterior, el término “expresion” es cualquier expresión del tipo de retorno del método (o compatible con el mismo). Cuando se ejecuta esta instrucción, se evalúa la expresión y el valor resultante es el que se devuelve como resultado de la

ejecución del método, que se da por terminada (aunque hubiera escritas a continuación otras instrucciones).

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

¿Puede una clase retornar una variable de un tipo estructurado, como un objeto de una clase?

¿Cuál es la razón que justifica que un constructor solo deba tener inicialización de atributos y no otras operaciones?



Conclusión

En esta segunda semana del curso se ha abordado otros temas asociados a la programación orientada a objetos. Se analizó la estructura de una clase, el concepto de atributo y los tipos de datos posibles. Al mismo tiempo se analizó el concepto de método, los parámetros que pueden recibir y su valor de retorno.

Otros temas que han sido abordados en esta ocasión son los accesorios y mutadores, que corresponden a métodos que permiten obtener el valor de un atributo y modificarlo, respectivamente. También se revisó el concepto de constructor, y su importancia en la inicialización de objetos.

Las siguientes tareas dentro de esta semana son poner en práctica los conceptos que se han visto acá, usando Greenfoot como plataforma de desarrollo. Por el momento es necesario plantearse las siguientes interrogantes:

- Además de las declaraciones de variables en un método, ¿qué otras acciones se podrían realizar?
- ¿Es obligatorio que en un constructor inicialice todos los atributos de una clase?

Bibliografía

- [1] Flórez Fernández, H. A. (2012). Programación orientada a objetos usando java. Bogotá, Colombia: Ecoe Ediciones.
- [2] Prieto Saez, N. y Casanova Faus, A. (2016). Empezar a programar usando Java (3a. ed.). Valencia, Spain: Editorial de la Universidad Politécnica de Valencia.
- [3] PC Resumen, “La clase Object en Java”. Referencia: <https://www.pcrresumen.com/menu-software/35-lenguajes-de-programacion/java/71-la-clase-object-en-java>
- [4] IBM, “Cómo empezar con los métodos nativos en Java”. Referencia: <https://www.ibm.com/docs/es/i/7.1?topic=interface-getting-started-java-native-methods>

